

Generating Representative Microservice Macrobenchmarks with Palette

Ali Fahad Khan Lodhi*
MPI-SWS

Vaastav Anand*
MPI-SWS

Matheus Stolet*
MPI-SWS

Jonathan Mace
Microsoft Research

Antoine Kaufmann
MPI-SWS

1 Introduction

Modern cloud systems at large companies are developed as microservice systems [2–6]. Validating and testing new advancements for microservices often requires developers to experiment with interventions—that is change aspects of the system with new design decisions, architectural choices, algorithms, backend components, and other strategies—in order to estimate and assess their impact on the system.

Ideally, developers and researchers would execute these intervention experiments on production systems to make claims about the technique’s scalability and generalizability. However, access to production systems is limited to a select few. Even if one could procure access, the scope of experimentation is limited to minimize disruptions.

Despite the limited access to production systems, distributed traces are often readily available and researchers can use them to glean insight into these systems. Distributed traces capture rich structural and temporal information about the execution of the system, such as latency, execution patterns, branch probabilities, and call probabilities.

We posit that distributed traces can support general purpose intervention experiments given the rich volume of system behavior they capture. However, we believe that currently there are three key challenges that prevent distributed traces from being used for intervention experiments. First, while distributed traces capture a variety of metrics, they are observations of the intrinsic behavior of the system. The system behavior that caused these observations is not explicitly captured in distributed traces. Thus, we need a mechanism that can use the distributed traces to derive a model of the intrinsic behavior of the system. Second, currently there is no mechanism to convert intervention experiments into targeted modifications of the system without modifying the rest of the system behavior. The fundamental tenet of an intervention experiment requires that all the other factors in the system must be held constant to isolate the impact of the intervention. Third, converting distributed traces to a runnable system for supporting general purpose intervention experiments is non-trivial as different intervention experiments might wish to preserve different characteristics.

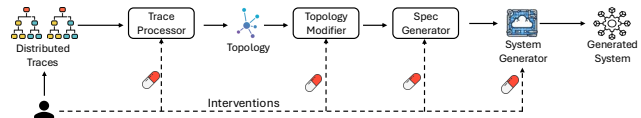


Figure 1: Palette Pipeline

To overcome these challenges, we are building Palette, a system designed for natively supporting intervention experiments using distributed traces. Palette provides a new abstraction called *system topology* that models system behavior captured by generating a graphical causal model (GCM). To support targeted modifications for interventions, Palette provides a set of primitive operations that allow developers to make localized changes in the system topology. Finally, Palette provides a generation mechanism for converting a system topology to a runnable system. Our solution leverages the GCM at runtime to model the causal dependencies from the original system and uses it to more faithfully sample metrics for execution behavior, such as the amount of work to be done in a service or the payload size to be used between two services given the current state of the system.

2 Palette Design Sketch

Figure 1 shows the design pipeline of Palette. Palette processes trace datasets to generate a system topology that encodes structural relationships, execution patterns, and performance characteristics of the observed system. This topology is converted into specifications that encodes mechanisms to ensure the preservation of the learned characteristics. Palette then uses Blueprint [1] to generate a full implementation of the system augmented with a GCM-based runtime that steers execution to reflect the operation of the original system.

System Topology. Palette generates a system topology from the statistics collected during trace processing; the topology encodes the calculated statistical information into an abstract representation which can be exposed to the user for further modification. It models the structural relationships within the system through a directed graph that encodes the caller-callee interactions between services and APIs, while a Probabilistic Finite Automaton (PFA) captures the diverse execution pathways of each API. The topology models the performance properties of the system by using a graphical

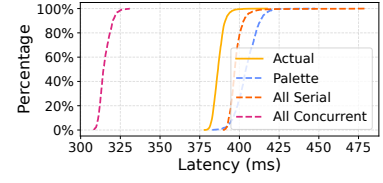
*Student Authors

causal model (GCM). Both structural and performance properties of the original system must be preserved to generate and execute a new representative system.

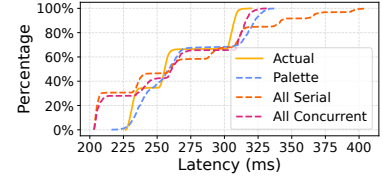
Generation mechanism. Palette processes the trace dataset to calculate statistical information about each service including a list of all its APIs, the execution time for each API, the probability of every outgoing call for each API, and the dependencies for each service. Palette then builds a PFA for each API and generates a causal graph for each API and generates a causal equation for each node in the causal graph. The causal equation is entirely dependent on the specific performance property being modeled by the GCM. Palette then fits a linear regression for the latencies using the observed trace data to find the coefficients used to estimate the causal effect of each parent on a given node in the graph. Palette then converts the topological model into actual source code that encodes the performance properties of the system. For each API, Palette converts its corresponding PFA into a set of local functions to encode PFA states and transitions. Palette also encodes a GCM model for the API and links it to the GCM-enabled runtime. Palette generates the source code in the form of Blueprint's input specifications to generate a full implementation of the system.

Supporting Interventions. Palette allows users to execute interventions at four different stages. First, users can modify the trace processing step to filter out invalid or irrelevant edge cases and include data dimensions in the GCM generation procedure. Second, the user can apply custom modifications to the topology through a set of modification primitives. For example, a researcher can add new edges and vertices to represent new causal relationships between microservices. Large topologies can also be downscaled to fit the physical testbed available to the researcher by removing edges and vertices representing the different APIs. Third, the user can change how a performance property is achieved. For example, to achieve desired latency, dependent on the use case, the user might want its services to sleep and not do any work or they might want the services to add work that is memory bound. Finally, users can add further interventions to the system by modifying the Blueprint IR to modify concrete service implementations. For example, the IR can be modified to support different RPC frameworks that change how the actual benchmark system gets instantiated.

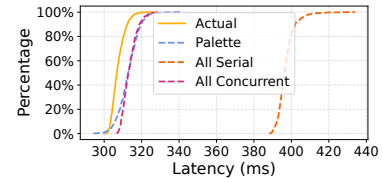
GCM-Based Runtime. Each API leverages the causal equation and coefficients produced by Palette to drive system execution in a way that resembles the original system behavior. During execution, systems may deviate from the expected behavior. GCMs enable live corrections during runtime when execution diverges from expected behavior. To do so, the runtime makes measurements for different nodes of the causal graph during the execution and uses these measurements



(a) Serial Downstream Calls



(b) Mutually Exclusive Downstream Calls



(c) Concurrent Downstream Calls

Figure 2: End-to-End Latency comparison between Palette, actual, and baseline systems.

into the local causal equations at every API to make predictions about the expected behavior and then tune the amount of work API does to correct the deviation. The causal graphs of some APIs may require measurements made higher up in the call chain. Palette uses baggage propagation [7] to propagate the required measurements to the downstream nodes executing the API. Palette injects the exact code for adding the causal data into the baggage and propagate it downstream along with the request as the dependency graph is known at generation time.

Preliminary Results. To evaluate the effectiveness of Palette, we first build a simple application that can be configured to issue downstream calls using three execution patterns: (i) serial, (ii) mutually exclusive, and (iii) concurrent. We run an open-loop workload generator against each configuration to collect execution traces, which are then used to generate replica systems using Palette and two baseline approaches. The baseline systems generate replicas that execute all downstream calls either serially or concurrently, regardless of the observed execution pattern. Figure 2 plots the end-to-end latency distributions for the original applications and the generated replicas for the serial and mutually exclusive configurations. The results show that each baseline preserves the latency distribution only for the execution pattern it assumes, whereas Palette accurately preserves the latency distributions in both configurations.

References

- [1] Vaastav Anand, Deepak Garg, Antoine Kaufmann, and Jonathan Mace. Blueprint: A toolchain for highly-reconfigurable microservice applications. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 482–497, 2023.
- [2] Adrian Cockcroft. The evolution of microservices. (April 2016). Retrieved October 2020 from <https://www.slideshare.net/adriancockcroft/evolution-of-microservices-craft-conference>, 2016.
- [3] Adrian Cockcroft. Microservices workshop: Why, what, and how to get there. (April 2016). Retrieved October 2020 from <https://www.slideshare.net/adriancockcroft/microservices-workshop-craft-conference>, 2016.
- [4] Einas Haddad. Service-oriented architecture: Scaling the uber engineering codebase as we grow. (September 2015). Retrieved October 2020 from <https://eng.uber.com/service-oriented-architecture/>, 2015.
- [5] Mazdak Hashemi. (January 2017). Retrieved February 2021 from https://blog.twitter.com/engineering/en_us/topics/infrastructure/2017/the-infrastructure-behind-twitter-scale.html, 2017.
- [6] Darby Huye, Yuri Shkuro, and Raja R Sambasivan. Lifting the veil on {Meta's} microservice architecture: Analyses of topology and request workflows. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 419–432, 2023.
- [7] Jonathan Mace and Rodrigo Fonseca. Universal context propagation for distributed system instrumentation. In *Proceedings of the thirteenth EuroSys conference*, pages 1–18, 2018.